Project Report: Real-Time Financial Fraud Detection using Machine Learning & Stream Processing

Course: CPS 698 **Mentor:** Dr. Sisheng Liang **Team 3 Members:** Vinay Kumar Simma, Pradeep Yarlagadda, Praneeth Reddy Ambati **Date:** May 3rd, 2026

I. Introduction

Background and Context

The rapid digitization of the banking and fintech sectors has precipitated an unprecedented volume of online financial transactions. Consequently, malicious actors have developed increasingly sophisticated methods to execute fraudulent activities. Traditional rule-based fraud detection systems are no longer sufficient to combat these threats, as they are rigid, easily bypassed, and lack the ability to adapt to novel fraud patterns. Modern financial institutions require dynamic, intelligent systems capable of analyzing transactions and predicting fraudulent intent instantaneously.

Statement of the Research Problem

While machine learning classification models possess high accuracy in identifying fraud, executing these classifications on massive, continuous data streams presents a significant technical challenge. The core research problem addressed in this project is the **latency in large-scale classification**. Processing millions of transactions sequentially through a heavy machine learning model often results in bottlenecks, preventing real-time intervention and allowing fraudulent transactions to clear before they are flagged.

Objectives and Scope

The primary objective of this project is to architect and deploy a highly scalable, real-time streaming machine learning pipeline. The scope encompasses training an optimal classification model on historical financial data and integrating it into an event-driven streaming architecture capable of ingesting, classifying, and reporting simulated live transactions with millisecond latency.

Significance of the Work

By demonstrating a successful integration of high-performance ML models (XGBoost) with robust stream processing engines (Apache Kafka), this project provides a blueprint for financial institutions to drastically reduce fraud detection latency. It illustrates how open-source, containerized technologies can be orchestrated to achieve enterprise-level security and operational awareness.

II. Methods

Dataset Description and Preprocessing

The model was trained and evaluated using the **PaySim dataset**, a widely recognized synthetic financial dataset designed specifically for mobile money fraud detection research. The dataset simulates 30 days of transactions, comprising millions of records with features such as transaction type, amount, origin account balance, and destination account balance.

Data preprocessing involved handling extreme class imbalance (where less than 1% of transactions are fraudulent). Feature engineering included calculating balance differentials (e.g., changes in origin and destination balances before and after the transaction) and encoding categorical transaction types to

optimize them for tree-based algorithms. Due to cloud infrastructure constraints during deployment, an optimized 50,000-row subset of the data was utilized for live streaming demonstrations.

Machine Learning Architectures

The core predictive engine is built using **XGBoost (Extreme Gradient Boosting)**. XGBoost was selected over alternative models (such as Logistic Regression or deep neural networks) because tree-based algorithms inherently manage imbalanced datasets effectively without requiring aggressive synthetic oversampling (e.g., SMOTE). Furthermore, XGBoost is highly optimized for fast inference, allowing it to execute predictions in milliseconds, which is critical for the real-time API. A **Random Forest Classifier** was also evaluated as a baseline comparative model.

Real-Time Streaming Architecture

To achieve low-latency processing, a decoupled, event-driven streaming architecture was engineered:

1. **Apache Kafka & Zookeeper:** Acts as the central nervous system. A Kafka Producer simulates banking nodes by streaming transactions into a `fraud_transactions` topic.
2. **FastAPI Serving Layer:** The trained XGBoost model (`xgb_model.json`) is served via a FastAPI REST endpoint (`/predict`), completely decoupling model inference from data ingestion.
3. **Kafka Consumer & SQLite:** A Python-based Kafka Consumer continuously reads incoming transactions from the broker, invokes the FastAPI endpoint for predictions, and persists the evaluated records into an SQLite database. To support high concurrency between the Consumer (writing) and the Dashboard (reading), SQLite was configured with **Write-Ahead Logging (WAL)** mode.
4. **Streamlit Dashboard:** An interactive front-end application continually queries the SQLite database to display live operational metrics, recent alerts, and real-time fraud rates.

III. Results

Model Performance

The XGBoost model demonstrated exceptional performance on the validation dataset. The model successfully identified complex, non-linear fraud patterns, achieving an area under the ROC curve (ROC-AUC) exceeding 0.98.

Evaluation Metrics

The final evaluation yielded strong predictive metrics:

- **Precision:** High precision ensured that false positives were minimized, preventing legitimate users from experiencing blocked transactions.
- **Recall:** High recall verified that the vast majority of actual fraudulent transactions were successfully intercepted by the model.
- **Accuracy:** Overall accuracy remained consistently near 99%, though ROC-AUC was prioritized due to the imbalanced nature of the data.

System Latency and Operational Throughput

During live streaming tests on AWS, the architecture exhibited remarkable throughput. The decoupled FastAPI model evaluated incoming HTTP requests in sub-10 milliseconds. The end-to-end latency—from the moment the Producer published a transaction to the moment it appeared on the Streamlit dashboard as a

processed alert—averaged under 50 milliseconds, successfully fulfilling the real-time objectives of the research.

IV. Application

Real-World Deployment Scenario

To prove real-world viability, the entire pipeline was successfully deployed to the public cloud using an **AWS EC2 instance (Ubuntu 22.04)**. The deployment utilized **Docker** to containerize the Apache Kafka and Zookeeper clusters, guaranteeing perfect environment parity between local development machines and the production server. The FastAPI backend, Kafka Consumer, and Streamlit Dashboard were executed as resilient background processes (`nohup`).

Scalability and Implications

The architectural choice of using Apache Kafka ensures massive horizontal scalability. In a true enterprise banking environment, the single Kafka broker can be expanded into a multi-node cluster, and the FastAPI service can be load-balanced across Kubernetes pods to handle tens of thousands of transactions per second. This framework demonstrates that banks can implement highly scalable, open-source fraud detection systems without relying exclusively on expensive, proprietary SaaS platforms.

V. Discussion

Pipeline Robustness and Challenges Faced

The pipeline proved highly robust during sustained streaming tests; however, several significant technical challenges were overcome during development:

1. **Database Concurrency Locks:** Initially, the heavy write-load from the Kafka Consumer caused "database locked" errors when the Streamlit dashboard simultaneously attempted to read data. This was resolved by migrating SQLite to Write-Ahead Logging (WAL) mode.
2. **Explainability vs. Latency Trade-offs:** While SHAP (SHapley Additive exPlanations) values were used extensively during training to interpret model decisions, calculating them during live stream inference caused unacceptable API latency. Consequently, SHAP generation was intentionally removed from the real-time production endpoint to maintain millisecond response times.
3. **Cloud Resource Constraints:** Deploying heavy ML dependencies alongside Kafka on an AWS Free-Tier `t3.micro` instance resulted in disk-space exhaustion. This required strategic dataset sampling and isolated virtual environments to successfully complete the deployment.

Limitations and Future Work

A primary limitation of the current system is its reliance on a static, pre-trained XGBoost model. In a live environment, fraud patterns evolve rapidly. Future work should involve implementing an active learning pipeline that periodically retrains the model on newly flagged transactions and dynamically hot-swaps the model weights in the FastAPI backend without incurring downtime. Additionally, migrating from SQLite to a distributed NoSQL database (e.g., Apache Cassandra) would further enhance the system's resilience for enterprise-scale deployments.

Lessons Learned

This project illuminated the profound differences between training a machine learning model in an isolated Jupyter Notebook and operationalizing it within a distributed system. The team gained critical hands-on

experience with stream processing (Kafka), microservice architecture (FastAPI), cloud deployment (AWS/Docker), and managing system-level concurrency.

Appendices

- **Appendix A:** Approved Project Proposal
- **Appendix B:** GitHub Repository: [Link to Repository]
- **Appendix C:** Weekly Meeting Minutes
- **Appendix D:** Final Project Self-Evaluation

(Note: The actual contents of the appendices are to be attached by the team prior to submission).

References

[1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *24th European Simulation and Modelling Conference*, 2016. [2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. [3] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," in *Proceedings of the NetDB*, 2011. [4] S. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.